MODULO 4 · LEZIONE 7

Skill: istruzioni che l'agente carica quando servono

Agenda

Blocco	Modalità	Slide
Cos'è una skill	teoria	3–4
Tre modi di dare istruzioni	teoria	5–6
Le skill nel coding agent	teoria e demo	7–10
La skill report-mensile	lavoro vostro	11–12
Rientro: cosa ha fatto l'agente	discussione	13–16
La stessa idea in un agente dell'SDK	codice spiegato	17–20
Skill o AGENTS.md	discussione	21–22



COS'È UNA SKILL

La definizione

> Le skill sono capacità modulari che estendono le funzioni dell'agente. Ogni skill raccoglie istruzioni, metadati e risorse facoltative (script, modelli) che l'agente usa da solo quando servono.

Istruzioni

Come si fa il lavoro, passo per passo.

Metadati

Nome e descrizione: cosa fa e quando usarla.

Risorse

Script da eseguire, modelli e documenti da leggere se servono.



COS'È UNA SKILL

Cosa sa il modello di una skill

Nella prima chiamata di Claude Code, in Modulo 4 · Lezione 1, c'era l'elenco di 16 skill: 7.397 caratteri, una riga per skill.

> Tre righe dell'elenco, dalla trace
>
> ```
> - init: Initialize a new CLAUDE.md file with codebase documentation
> - security-review: Complete a security review of the pending changes on the current branch
> - simplify: Review the changed code for reuse, simplification, efficiency, and altitude cleanups, …
> ```

Il modello vede il nome e la descrizione. Da qui decide se una skill serve alla richiesta.

Le istruzioni restano in un file sul disco. Il modello le legge solo se decide di usare la skill.



TRE MODI DI DARE ISTRUZIONI

Tre file di istruzioni per il coding agent

Tutti e tre sono testo che finisce nel contesto del modello. Cambiano chi li scrive e quando entrano.

File	Chi lo scrive	Quando entra nel contesto	Nel progetto Valtesa
`AGENTS.md`	voi	sempre, per intero, all'inizio	venv, modello, cosa non toccare
`SKILL.md`	voi, o la scaricate	nome e descrizione sempre; il resto quando serve	come si prepara il report del mese
`memoria`	il coding agent	all'inizio, l'indice dei ricordi salvati	«i test si lanciano dal venv del modulo»

> Oggi le skill. La memoria su file è un approfondimento di Modulo 4 · Lezione 13.



TRE MODI DI DARE ISTRUZIONI

Cosa sta sempre nel contesto

Misurato con /context di Claude Code nella cartella working/ di oggi, prima di qualsiasi richiesta.

1.400

token: AGENTS.md, a ogni chiamata

100

token circa: una skill nell'elenco (70 e 110 le due skill di oggi)



LE SKILL NEL CODING AGENT

Una skill vera: webapp-testing

Dalla repository anthropics/skills su GitHub: prova un'applicazione web nel browser, con Playwright. Una skill è una cartella con un file SKILL.md.

```
.claude/skills/webapp-testing/
├── SKILL.md              le istruzioni, 3.913 caratteri
├── scripts/with_server.py  avvia il server
└── examples/               tre esempi di codice
```

```
---
name: webapp-testing
description: Toolkit for interacting with and testing local
web applications using Playwright. …
---
# Web Application Testing
```

name: minuscole, cifre e trattini, uguale al nome della cartella.

description: cosa fa e quando usarla, al massimo 1.024 caratteri. Nel corso, su una riga.

Il corpo: la procedura in Markdown. Accanto, se servono, script, documenti e modelli.

> Lo stesso formato vale in più coding agent:
>
> **Claude Code** → `.claude/skills/`
>
> **Codex** → `.agents/skills/`
>
> **Cursor** → `tutte e due`

La lettura progressiva

L'intestazione di SKILL.md

webapp-testing: Toolkit for interacting with and testing local web applications using Playwright. Supports verifying frontend functionality, debugging UI behavior, capturing browser screenshots, and viewing browser logs.

Il corpo di SKILL.md

To test local web applications, write native Python Playwright scripts. … Always run scripts with --help first to see usage. DO NOT read the source until you try running the script first …

Gli altri file della cartella

scripts/with_server.py, 3.693 caratteri · examples/, tre file

1 · Sempre

Una riga nell'elenco delle skill, a ogni chiamata: circa 70 token.

2 · Quando serve

La richiesta corrisponde: il modello chiama Skill e riceve il corpo senza l'intestazione, 3.574 caratteri, una volta.

3 · Se la procedura lo dice

Uno script si esegue e ne entra solo l'output. Se non serve, resta sul disco.



Il contesto durante una sessione

«Apri nel browser output/report-2026-07.html, controlla che ci siano le tabelle e il grafico e salvami uno screenshot.» Ogni colonna è il contesto in quel momento.

1 · Inizio

- prompt e tool · 24.200 token
- AGENTS.md · 1.400
- elenco skill · 2.400, webapp-testing 70

17 skill nell'elenco, una riga ciascuna.

2 · Richiesta

- prompt, tool, AGENTS.md
- elenco skill
- la richiesta: browser, screenshot

La richiesta non nomina la skill.

3 · Skill

- prompt, tool, AGENTS.md
- elenco skill
- la richiesta
- corpo di webapp-testing · 3.574 caratteri

`Skill(webapp-testing)`

4 · Lavoro

- prompt, tool, AGENTS.md
- elenco skill
- la richiesta
- corpo · 3.574
- output dello script Playwright · 400 caratteri

with_server.py ed examples/ non servono: restano sul disco.



DEMO DEL DOCENTE

webapp-testing sul report di luglio

Il docente installa la skill di Anthropic nella sua cartella `working/` e la fa usare a Claude Code. Voi guardate.

1 Installare

La cartella skills/webapp-testing della repository di Anthropic, copiata in .claude/skills/.

2 Elenco

Una sessione nuova, /skills e /context: la skill c'è, circa 70 token.

3 Richiesta

«Apri nel browser output/report-2026-07.html … salvami uno screenshot.»

4 Risultato

Skill(webapp-testing), uno script Playwright, lo screenshot in output/.

⚠ **Da guardare:** i tre livelli della lettura progressiva, uno dopo l'altro, e lo screenshot del report, aperto alla fine.



LA SKILL REPORT-MENSILE

Cosa deve fare la skill dell'esercizio

Ogni mese l'ufficio del personale vuole il report delle note spese: la pagina HTML di report.py e una sintesi di poche righe.

Le note → `dati/note_spese.csv`, e gli export che arrivano dal gestionale, `dati/export-*.csv`: file CSV nella cartella del progetto.

1 Trova i dati

Uno script dice in quali file c'è il mese e cosa manca.

2 Fa il report

Con report.py; se non basta, lo adatta con un test.

3 Controlla

Le note e il totale del report sono quelli dello script.

4 Scrive la sintesi

Sei righe per l'ufficio, con cosa va verificato.

> Non è una lista di script da lanciare: l'agente legge cosa esce e, se i dati non tornano, cerca il perché, cambia lo script o vi chiede. Lo provate su luglio, il caso semplice, e su settembre, un caso più difficile.



LAVORO VOSTRO

La skill report-mensile, con il coding agent

File: `working/specifica-skill.md`, poi `working/.claude/skills/report-mensile/`, che crea il coding agent. Comandi e richieste esatte sono nella dispensa: «Esercizio».

1 Preparare

Branch l07-lavoro, venv e chiave come sempre: pytest dice 85 passed.

2 Descrivere

In specifica-skill.md scrivete la descrizione della skill e i criteri di accettazione.

3 Costruire

Il coding agent crea la skill e il suo script dalla specifica, con un piano prima. Poi un commit.

4 Usare

Sessioni nuove: il report di luglio, poi quello di settembre. Guardate cosa fa l'agente.

5 Controllare

Aprite report e sintesi, leggete il diff, pytest; una richiesta estranea non carica la skill.

Fatto quando ci sono report e sintesi di luglio e di settembre, avete letto nel diff cosa ha cambiato negli script, pytest è verde e avete scritto cosa ha fatto caricare la skill. Poi un commit.

Se la skill non si carica: copiate quella del docente da ripartenza/ e ripartite dal passo 4.



RIENTRO

Quando i dati non tornano: cosa fa l'agente

Settembre è il mese con i dati incompleti. Lanciato da solo, `python report.py 2026-09` scrive un report di 0 note, senza errori. Una sessione con la skill, passo per passo:

1 Cerca i dati

dati_del_mese.py: settembre non è in note_spese.csv, è nell'export appena arrivato: 190 note, senza la colonna notti.

2 Adatta lo script

Aggiunge a report.py un'opzione per leggere l'export, con il suo test.

3 Legge l'errore

report.py si ferma: Bologna non ha massimali. Il report deve reggere il caso.

4 Legge la policy

La skill gli indica l'art. 3: per le altre città i massimali stanno nell'autorizzazione della trasferta.

5 Controlla i numeri

Report e script dicono le stesse 190 note e 9.934,75 €.

6 Scrive la sintesi

Con cosa verificare: 16 note di Bologna non controllate, 52 hotel senza notti.

> Gli script fanno i conti; l'agente legge l'output e cambia strada quando qualcosa non torna.



RIENTRO

Quando i dati non tornano: con e senza la skill

La richiesta del report di settembre, il mese con i dati incompleti: 5 sessioni nuove senza la skill, 5 con la skill.

Nelle 5 sessioni	Senza skill	Con la skill
trova i dati nell'export	5	5
report di 190 note	2	4
sintesi per l'ufficio	0	4
spiega Bologna con l'art. 3 della policy	0	5
segnala gli hotel senza notti	1	5

> Il modello è lo stesso e ragiona anche senza skill. La skill gli dice cosa consegnare e dove guardare quando i dati non tornano.



RIENTRO

Nome e descrizione decidono se la skill si apre

Ho cambiato solo nome e descrizione della mia skill e ho fatto la stessa richiesta in 5 sessioni nuove: quante volte il coding agent la apre?

Nome e descrizione	Richiesta	Aperta	Cosa si vede
report-mensile, con la descrizione completa: cosa fa e quando usarla	il report di luglio	5 su 5	nome e descrizione chiari: si apre
report-mensile, con «Lancia report.py e dati_del_mese.py per un mese.»	il report di luglio	5 su 5	la descrizione è povera, ma il nome basta
procedura-a, con «Appunti del progetto.»	il report di luglio	0 su 5	tutti e due vaghi: il report si fa senza procedura
report-mensile, con la descrizione completa	«Spiegami cosa fa rimborsi.py»	0 su 5	richiesta che non c'entra: giusto così

> Se né il nome né la descrizione dicono a cosa serve la skill, il modello non la apre: come per i tool di Modulo 3 · Lezione 7.



RIENTRO

La vostra specifica e quella del docente

Aprite la vostra specifica-skill.md e confrontatela con questa: prima la descrizione, poi i criteri di accettazione.

La descrizione del docente

> Prepara il report mensile delle note spese per l'ufficio del personale di Valtesa - la pagina HTML con totali, sedi, voci e note fuori massimale, e una sintesi di poche righe da mandare all'ufficio, con i numeri controllati. Da usare quando si chiede il report, il riepilogo o i numeri delle note spese di un mese.

I criteri del docente

1. «Il report di luglio», in una sessione nuova, carica la skill e lancia dati_del_mese.py.
2. Luglio: il report dice 164 note e 8.963,53 €, la sintesi ha al massimo sei righe.
3. Settembre: report di 190 note dall'export, report.py cambiato con un test, sintesi con Bologna e le notti.
4. Un mese che non c'è: si ferma e lo dice, senza creare file.
5. Una richiesta estranea non la carica; dati/ non cambia; pytest è verde.

Quale criterio vi mancava? Quale avete in più?



CODICE SPIEGATO

La stessa idea in un agente dell'SDK

L'assistente di Valtesa riceve delle skill sue, con la stessa lettura progressiva del coding agent. Il codice è già nel file: il docente lo commenta, voi seguite.

1 Aprire

working/skill_agente.py e working/skill-assistente/rimborso-trasferta/SKILL.md.

2 Seguire

Il docente commenta il codice sul suo schermo, voi lo seguite sul vostro file. Non si ricopia niente.

3 Provare

Dopo, da soli: python skill_agente.py --istruzioni stampa il testo che riceve il modello.

Fatto quando sapete dire cosa riceve il modello all'inizio e cosa riceve dopo aver chiamato apri_skill.

LA STESSA IDEA IN UN AGENTE DELL'SDK

Nelle istruzioni, solo l'indice

indice() legge l'intestazione di ogni SKILL.md in skill-assistente/ e fa una riga per skill, come l'elenco di Claude Code.

```
SKILL = CARTELLA / "skill-assistente"

def indice():
    righe = []
    for file in sorted(SKILL.glob("*/SKILL.md")):
        campi, _ = leggi_skill(file)
        righe.append(f"- {campi['name']}: {campi['description']}")
    return "\n".join(righe)

ISTRUZIONI_SKILL = f"""… il testo a destra … {indice()}"""
```

Il testo aggiunto alle istruzioni

Hai delle skill: istruzioni scritte per alcuni compiti. Qui sotto c'è solo l'elenco, con nome e descrizione. Se la domanda corrisponde alla descrizione di una skill, prima di tutto aprila con apri_skill e segui le sue istruzioni. Altrimenti non aprirla.

- rimborso-trasferta: Prepara le indicazioni per una trasferta di lavoro - quanto si può spendere per pasti, albergo e taxi …



LA STESSA IDEA IN UN AGENTE DELL'SDK

Il corpo, con un tool

apri_skill fa per l'assistente quello che il tool Skill fa per Claude Code: rimanda il corpo di una skill, solo quando il modello lo chiede.

```python
@function_tool
def apri_skill(nome: str) -> str:
    """Apre una skill dell'elenco e rimanda le sue
    istruzioni, da seguire per rispondere. …"""
    file = SKILL / nome / "SKILL.md"
    if Path(nome).name != nome or not file.exists():
        return f"Nessuna skill «{nome}». Le skill sono:\n{indice()}"
    return leggi_skill(file)[1]   # solo il corpo

agente = Agent(...,
    instructions=assistente_base.instructions + ISTRUZIONI_SKILL,
    tools=assistente_base.tools + [apri_skill], model=MODELLO)
```

La traccia: «Vado a Milano il 14 luglio per due notti…»

`apri_skill(rimborso-trasferta)`

`leggi_file(massimali.csv)`

`cerca(autorizzazione) · cerca(nota spese)` …

→ Trasferta a Milano, dal 14 luglio, 2 notti: pranzo 25 €, cena 55 €, taxi 40 €, hotel 150 € a notte (300 €) …



LA STESSA IDEA IN UN AGENTE DELL'SDK

Quando gpt-6-luna apre la skill

skill_agente.py, 10 giri per domanda, ognuno in una sessione nuova.

10 su 10

«Vado a Milano il 14 luglio per due notti: quanto posso spendere…?»

La apre, e risponde nel formato della skill con i massimali giusti. Lo stesso per una notte a Roma.

0 su 10

«Come chiedo un giorno di ferie?»

Non la apre: la domanda non corrisponde alla descrizione.



SKILL O AGENTS.MD

Dove va un'istruzione

Si decide con due domande: chi la legge, il coding agent o l'assistente? Serve a ogni richiesta o solo per un compito?

Chi la legge	Serve sempre	Serve per un compito
il coding agent	AGENTS.md: il venv, il modello, cosa non toccare	una skill in .claude/skills/: il report del mese, provare una pagina nel browser
l'assistente	ISTRUZIONI in assistente.py: chi è, come cita i file	una skill in skill-assistente/: le indicazioni per una trasferta

> Un'istruzione sta in un posto solo. Se la stessa regola è in AGENTS.md e in una skill, prima o poi i due file dicono cose diverse.



SKILL O AGENTS.MD

Quattro istruzioni da sistemare

Per ognuna, a voce: AGENTS.md, una skill del coding agent, le ISTRUZIONI dell'assistente o una skill dell'assistente?

1

«I test si lanciano con pytest dalla cartella del progetto, prima e dopo ogni modifica.»

2

«Per i giorni di ferie che restano: il regolamento ferie, poi le assenze in assenze.csv, poi il conto.»

3

«Prima di consegnare il lavoro: pytest, il diff riletto, i criteri di accettazione controllati uno per uno, il commit.»

4

«Cita i file che usi tra parentesi quadre, per esempio [faq-portale-dipendenti.md].»



Grazie

La prossima lezione: MCP, tool che girano in un altro processo (Modulo 4 · Lezione 8).

Prima: un commit sul branch l07-lavoro, poi git switch main e git pull.